



# The Network Automation Map

Navigate Your Journey with the NAF Framework

**Christian Adell**

Coreweave

**Damien Garros**

OpsMill

AUTOCON5

# About Us

## Christian Adell

Staff Engineer  
Coreweave



## Damien Garros

Co-founder and CEO  
OpsMill



# Introduction



**Modern networks demand automation that teams can trust, understand, and safely evolve.**



Yet, it's still overly complicated for most organizations

Building it is hard, maintaining it is almost impossible



# The pattern we keep seeing

## Rebuild from scratch

Teams keep recreating the same building blocks—data models, pipelines, integrations

## Learn the hard way

Most “best practices” are discovered through outages, drift, and painful rewrites instead of repeatable patterns.

## No shared blueprint for what ‘good’ looks like

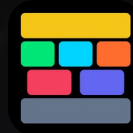
Without a common map, it’s hard to assess maturity, compare architectures, or even agree on what problem to solve first.





**Multiple people from the NAF community got together to find a solution**

**It quickly turned into a working group with weekly meetings**





This doesn't  
make any sense!

माह कवाला हैं!

On ne peut  
pas faire ça!

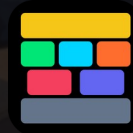
هذا لا يمكن

和 s て5 りない!

No es correcto!

خطن امان

It was hard at first  
Everyone was biased by their own tools and habits  
**Ironically, we were missing a shared lexicon**





**Collector**

**Orchestration**

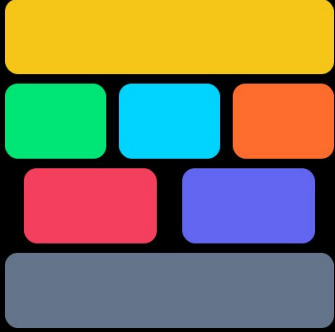
**Intent**

**Executor**

**Presentation**

**The solution was to stop focusing on tools and focus on functional building blocks**

**Quickly, everyone started talking the same language**

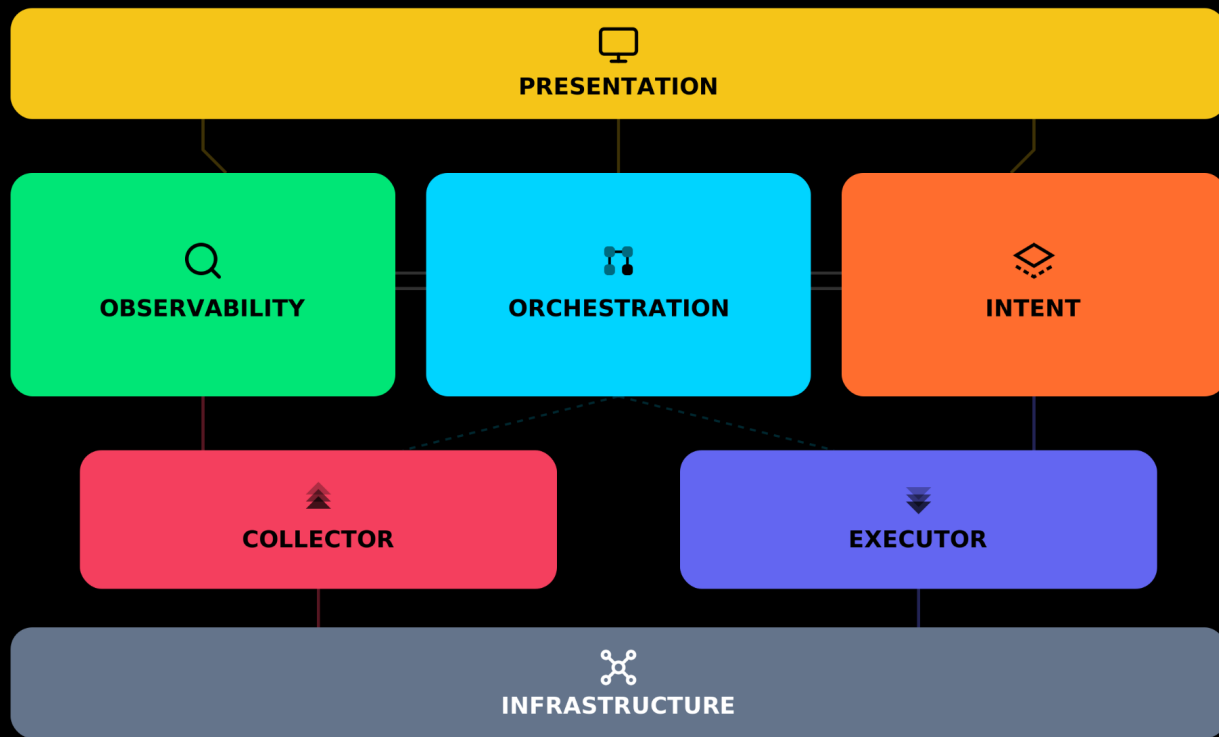


Simple building blocks that  
underpin any network  
automation framework

# NAF Framework

Not a reference architecture  
— it's a shared vocabulary +  
responsibilities.

# NAF Framework



# NAF Framework Fundamentals

## Functional Building Blocks

The NAF Framework is meant to be tool agnostic to avoid comparing apple and orange.

Each functional building block can be mapped to one or more tools and each tool can map to multiple functional blocks.

## Living Document Not a Protocol

The NAF Framework is meant to evolve over time.

It's not a strict implementation that imposes a specific way of doing things and in many cases, not all components are required to get started.

## Both for Beginners and Experts

The NAF Framework initiative is focusing initially on laying out the main building blocks to lay the foundation.

In the future the goal is to go deeper into each block.



**A map to help navigate the terminology and understand how things are related to each other**



**Presentation**



**Observability**



**Orchestration**



**Intent**



**Collector**

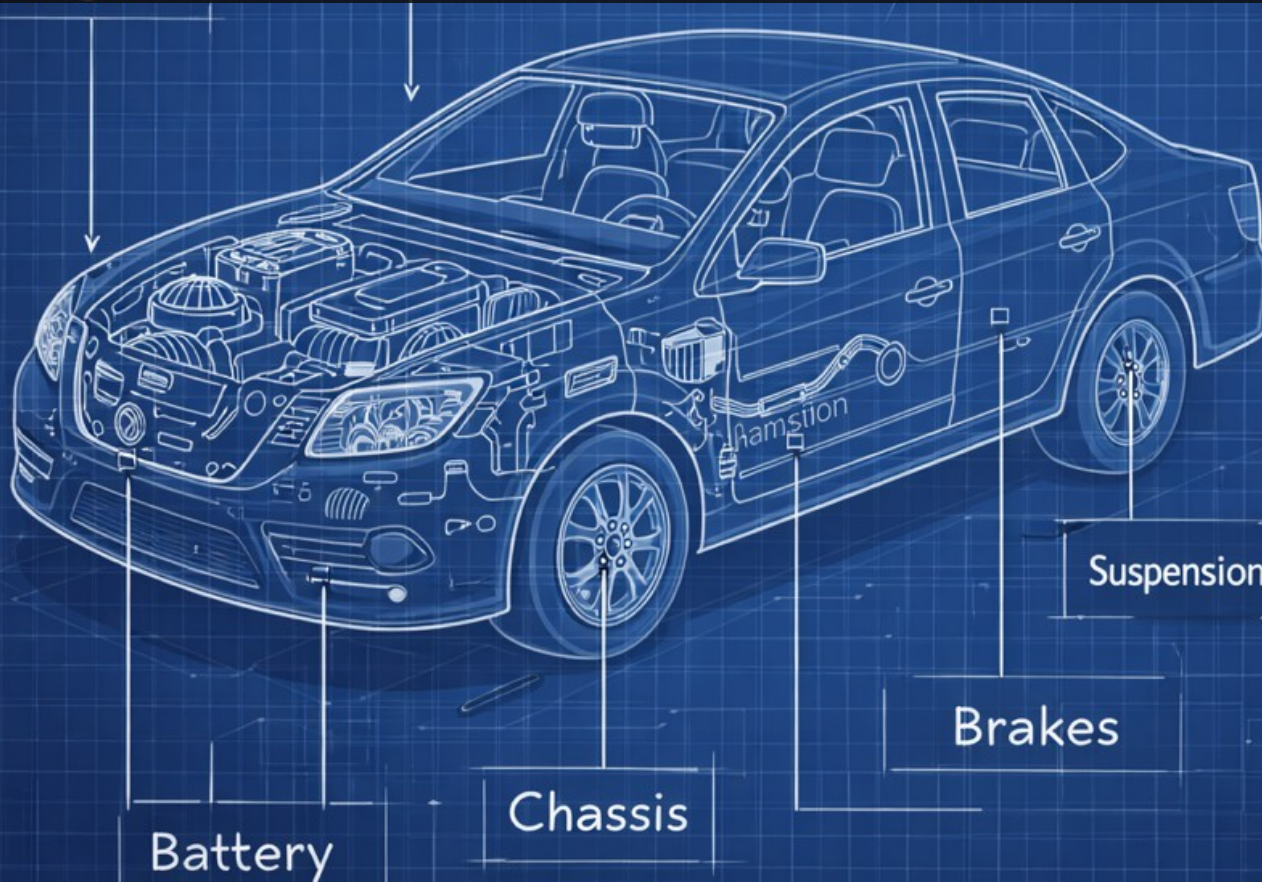


**Executor**



**Infrastructure**

# A functional blueprint to help everyone understand and build Network Automation Stack (system thinking)



## Electrical System

Battery, wiring, and electronics



## Safety

Airbags, seat belts, and crumple zones



# Target Audience

---

## Builders

Network Automation Engineer

Automation Architect

Software Engineer

Network Reliability Engineer

## Users

Network Engineer

Network Ops

Network Architect

Engineering Manager





**Wait, what about my tools ? Are they gone ?**



# Mapping between functional blocks and tools

|                                                                                                                          | PRESENTATION                                                                      | OBSERVABILITY                                                                       | COLLECTOR                                                                           |
|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  <b>gNMIc</b><br>Telemetry collection   |  |  |  |
|  <b>Prometheus</b><br>Metrics + alerts  |  |  |  |
|  <b>Grafana</b><br>Compliance dashboard |  |  |  |
|  <b>SuzieQ</b><br>State observability   |  |  |  |
|  <b>Kentik</b><br>Flow observability    |  |  |  |

Each tool / product / project can map  
**Completely or Partially**  
to  
**One or Multiple** blocks



# Where do I start?

---

## **Do I need to implement all blocks to get started?**

No, you should only implement what is required for your use case(s)

## **Is there a specific order to get started?**

No, it really depends on your use case(s).



**Ryan Shaw**



**Dinesh Dutt**



**Christian Adell**



**Claudia de Luna**



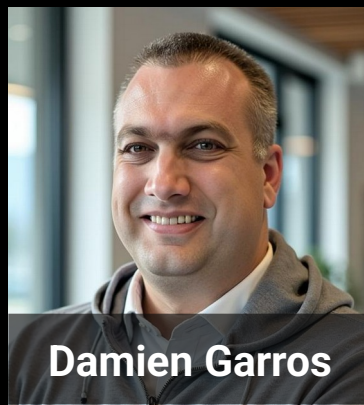
**Srividya Iyer**



**Wim Henderickx**



**Damien Garros**



**The NAF  
Community**



**The Team behind the Working Group**

# Practical Example



# Example- Roll out a new subnet across 50 branch sites

What we want to achieve, what the environment looks like, and the tools in play.

## THE GOAL

Deploy a new subnet **10.100.0.0/24** to every branch, validate the config on **every device**, and automatically **detect and remediate** any site that drifts out of compliance.

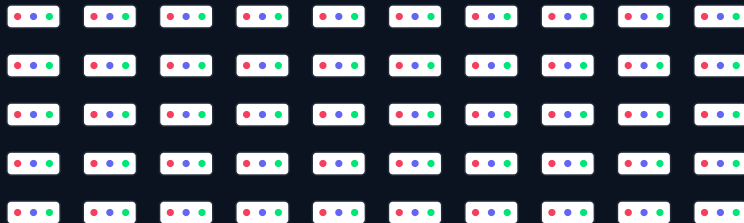
|            |                         |        |                      |
|------------|-------------------------|--------|----------------------|
| SUBNET     | 10.100.0.0/24           | SITES  | 50 branches          |
| DEVICES    | 1 router / site         | VENDOR | Mixed (NETCONF)      |
| VALIDATION | Interface + route table | SLO    | < 5 min drift detect |

## THE ENVIRONMENT

### HQ · DATA CENTER

Automation control plane

### 50 BRANCH SITES · 1 ROUTER EACH



S-01 ... S-50 · mixed-vendor, NETCONF-capable

## OUR STACK · 8 TOOLS



Infrahub  
INTENT · UI



AAP  
ORCHESTRATION



Ansible  
EXECUTOR



gNMIc  
COLLECTOR



Prometheus  
OBSERVABILITY



Grafana  
PRESENTATION



SuzieQ  
NET OBSERVABILITY



Kentik  
FLOW OBSERVABILITY



# Example - Mapping to the Framework

Rows are the 8 tools we run; columns are the 6 logic/transport blocks. Filled cell = tool covers that block. Infrastructure is excluded — it's the substrate, not a tool choice.

|                                                                                                                         | PRESENTATION | OBSERVABILITY | ORCHESTRATION | INTENT | COLLECTOR | EXECUTOR |
|-------------------------------------------------------------------------------------------------------------------------|--------------|---------------|---------------|--------|-----------|----------|
|  <b>Infrahub</b><br>Source of truth + UI | ●            | ●             | ●             | ●      | ●         | ●        |
|  <b>AAP</b><br>Runbooks + web UI         | ●            | ●             | ●             | ●      | ●         | ●        |
|  <b>Ansible</b><br>Config push (NETCONF) | ●            | ●             | ●             | ●      | ●         | ●        |
|  <b>gNMIc</b><br>Telemetry collection    | ●            | ●             | ●             | ●      | ●         | ●        |
|  <b>Prometheus</b><br>Metrics + alerts   | ●            | ●             | ●             | ●      | ●         | ●        |
|  <b>Grafana</b><br>Compliance dashboard  | ●            | ●             | ●             | ●      | ●         | ●        |
|  <b>SuzieQ</b><br>State observability    | ●            | ●             | ●             | ●      | ●         | ●        |
|  <b>Kentik</b><br>Flow observability     | ●            | ●             | ●             | ●      | ●         | ●        |

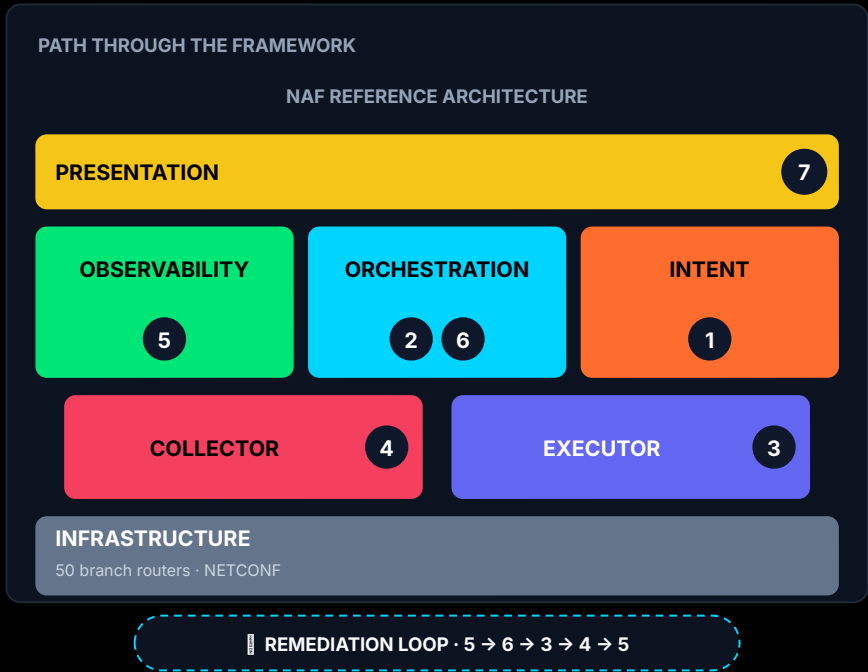
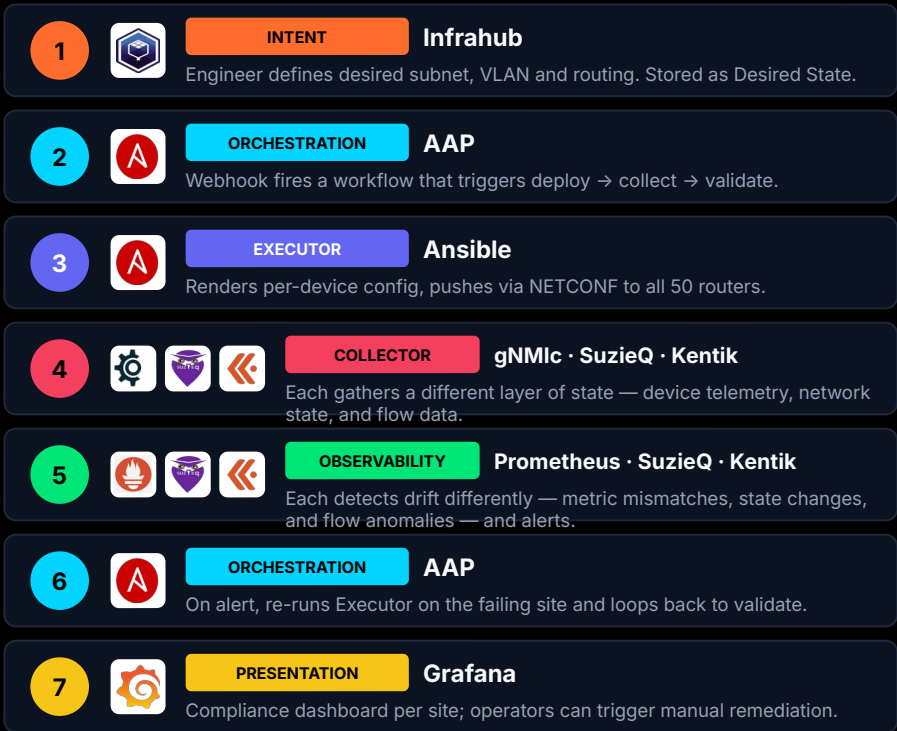
**One tool can span multiple blocks.** Infrahub = Intent + Presentation. AAP = Orchestration + Presentation. SuzieQ and Kentik each cover three blocks (Collector → Observability → Presentation) for different data types.

**Multiple tools can share one block.** Presentation is covered by 5 tools (Infrahub, AAP, Grafana, SuzieQ, Kentik). Collector is covered by 3 (gNMIc, SuzieQ, Kentik) — each gathering a different kind of state.



# Example - Step by Step

The seven steps in reading order on the left; the path they trace through the NAF building blocks on the right. Each block shows the step numbers that touch it.



# NAF Framework Deep Dive



# What defines a functional building block

A **tool-agnostic function** in the automation system —

a

**chunk of responsibilities** with clear goals, inputs/outputs, and capabilities

## Purpose / Goals

what outcome it exists to deliver

## Capabilities

expressed as **MUST** /

**SHOULD** / **MAY**

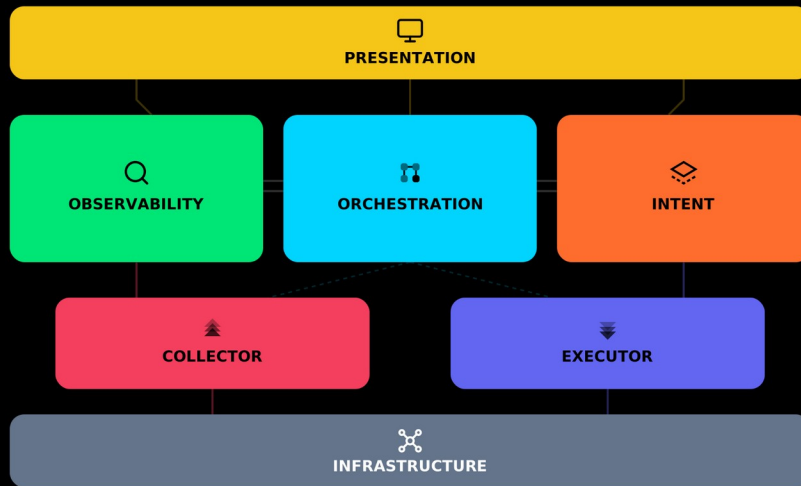
(requirements, not products)

## Boundaries

what it does and what it does not do (inputs → outputs)



# Intent



NAB Reference Architecture -- Minimal

*The authoritative store of what your network should look like (config, topology, services, and operational expectations) expressed at business level and translated into device-level artifacts.*

# Intent - Capabilities

## **MUST**

Represent any network-related aspect in structured form.

## **MUST**

Be exposed through a standardised, well-documented API.

## **SHOULD**

Include metadata that supports effective data governance.

## **SHOULD**

Provide a consistent and unified view of the desired state.

## **MUST**

Support create, read, update, and delete operations.

## **SHOULD**

Use vendor-neutral representation derived into device-specific artifacts.

## **SHOULD**

Be transactional and provide versioned access to data.

## **MAY**

Include all logic related to intended state management.

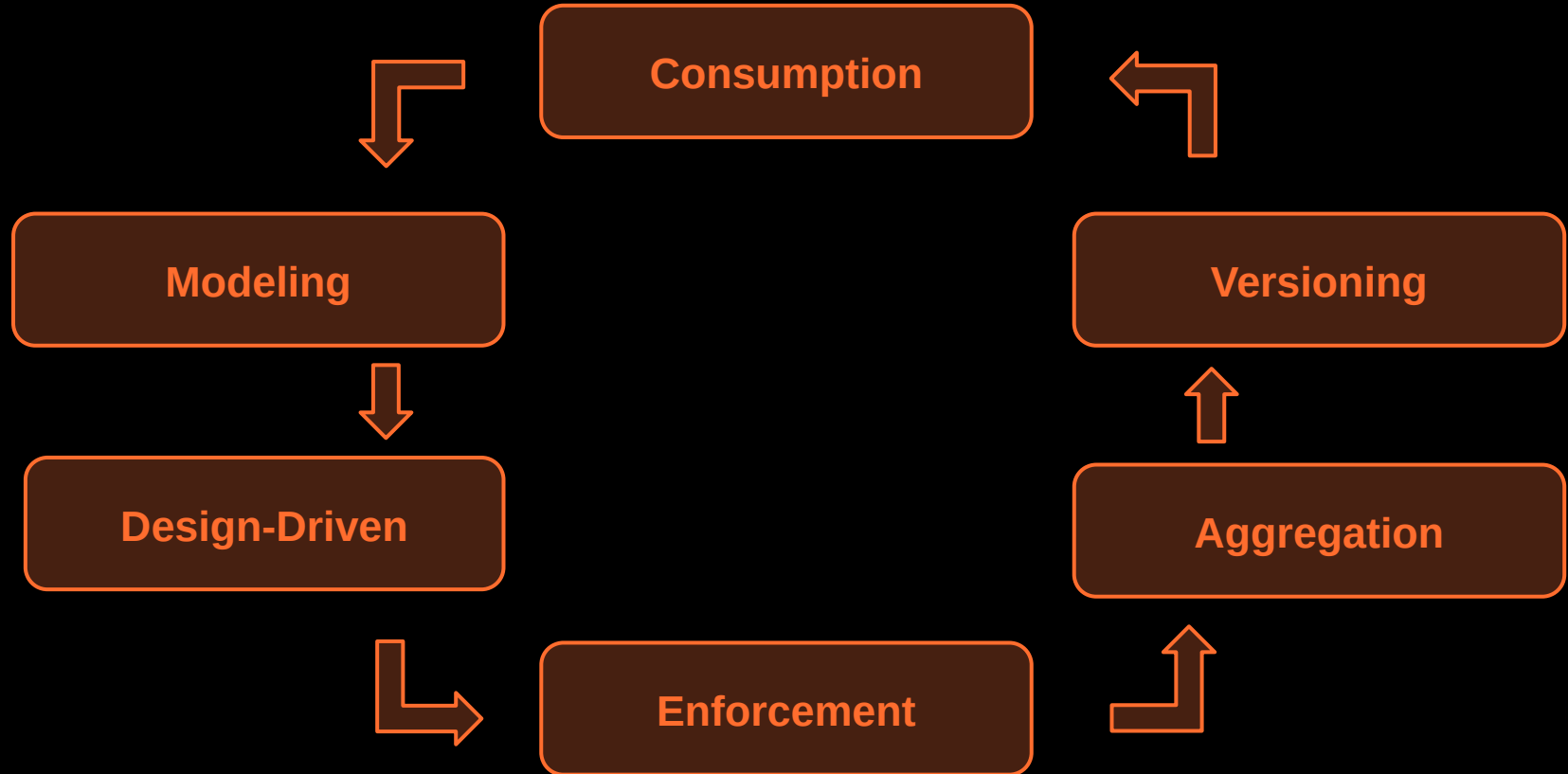


# Intent - Data Modeling

*What do you need to rebuild it  
if the network was completely lost?*



# Intent - Functionalities



# Intent - Functionalities (with description)

## Consumption

What data is needed by the other blocks? REST, GraphQL, gRPC, webhooks; context-aware views per consumer persona; AI-assisted interfaces.

## Modeling

Flexible schema(s) for all the data needed to re-create your network. Things like devices, interfaces, VLANs, circuits, services, etc.

## Design-Driven

Translates high-level intent into complete technical specs via templates and pattern libraries.

## Enforcement

Schema validation, uniqueness, referential integrity, business-rule checks; hard vs. soft enforcement tiers.

## Aggregation

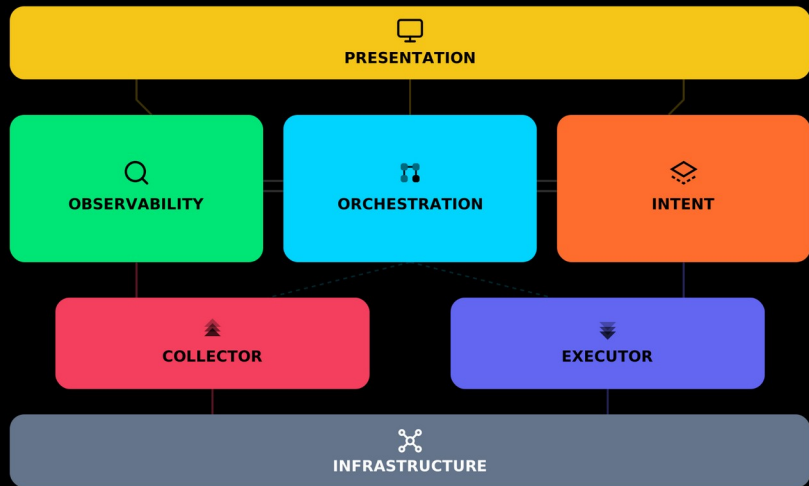
Federated sync from external systems; authority designation per data domain; conflict resolution policies.

## Versioning

Immutable audit trails, Git-like branching for parallel work, rollback, and atomic change transactions.



# Executor



NAD Reference Architecture -- Minimal

*The block that directly interacts with network devices to apply changes, translating intended state into actual configuration pushes, operational commands, and verifiable outcomes.*



# Executor - Capabilities

## **MUST**

Be capable of interacting with any supported network write interface.

## **SHOULD**

Derive execution input data from the intended state or its artifacts.

## **SHOULD**

Support any network operation that alters network state, not only config management.

## **SHOULD**

Provide a dry-run operation to check the expected result before applying.

## **SHOULD**

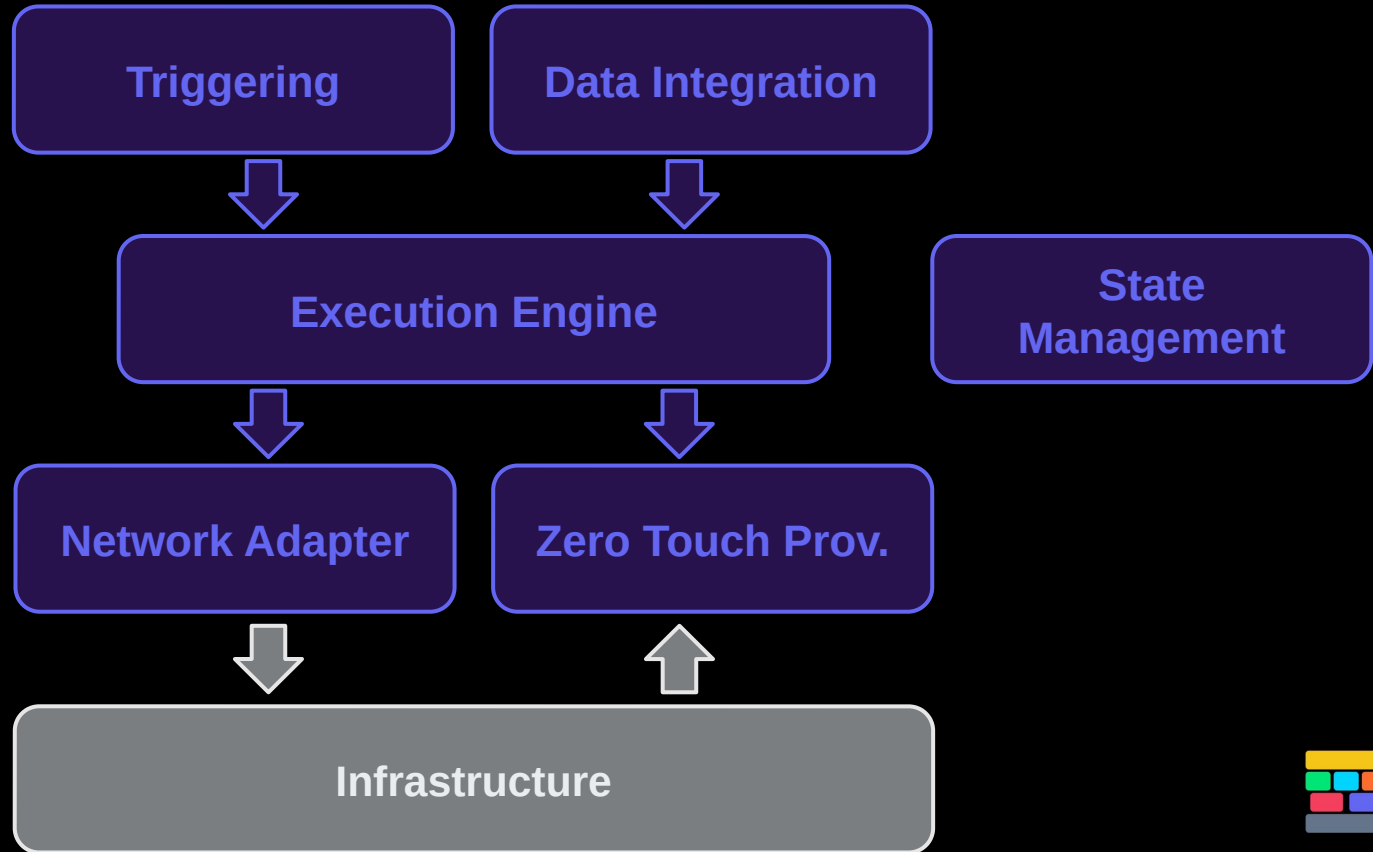
Support transactional execution of changes.

## **MAY**

Support both imperative and declarative approaches.



# Executor - Functionalities



# Executor - Functionalities (with description)

## Triggering

Synchronous (REST, webhook, RPC) and asynchronous (scheduler, event listener, message queue) entry points.

## Data Integration

Pulls inventory, credentials, and intent from the Intent block. Optionally queries observability for pre-check conditions.

## State Management

Persistent execution state per device; idempotency guarantees; locking to prevent concurrent conflicts.

## Execution Engine

Imperative and declarative modes; parallel/batched execution; retry with backoff; dry-run; partial-failure handling.

## Zero Touch Prov.

Fully unattended device onboarding triggered by network events, no human in the loop.

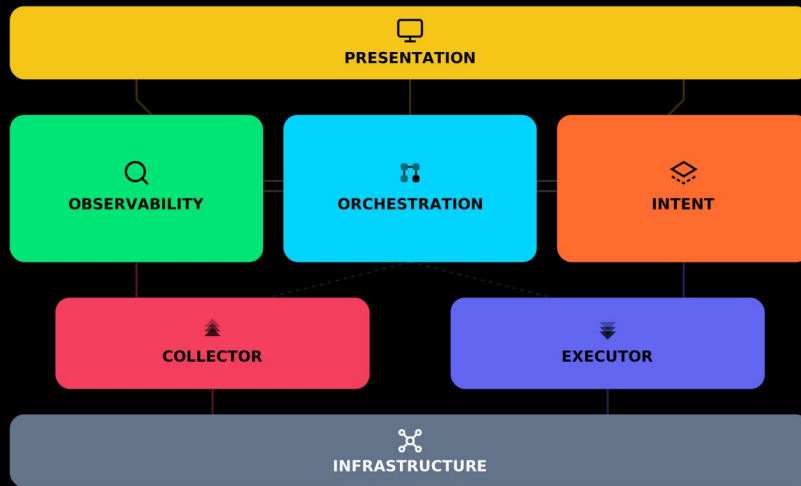
## Network Adapter

Protocol abstraction for SSH/CLI, NETCONF, gNMI, REST; connection pooling and response parsing.



# Network Infrastructure

*The block that defines the physical and virtual devices and services whose programmable interfaces, firmware quirks, and heterogeneity constrain every design decision above them.*



NMF Reference Architecture -- Minimal



# Network - Capabilities

## MUST

Expose at least one machine-readable management interface (SSH/CLI, NETCONF, gNMI, or REST API), ideally structured.

## SHOULD

Support idempotent configuration operations (applying the same config twice yields no unintended change).

## SHOULD

Provide an emulated environment representative of the production fleet.

## SHOULD

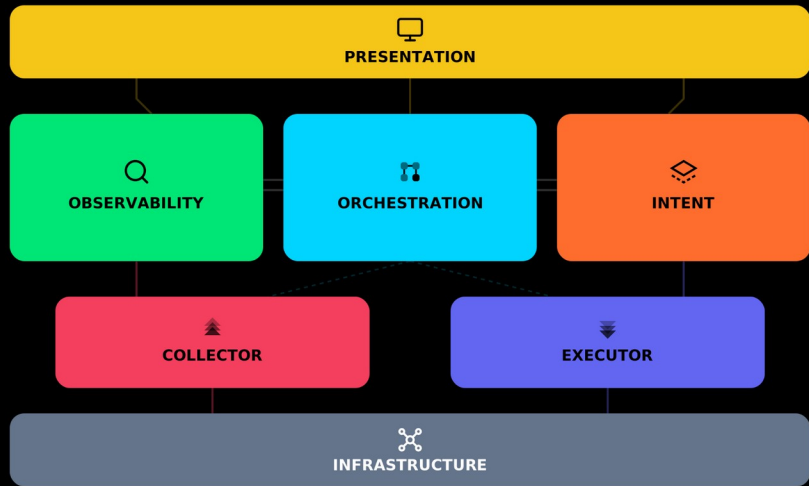
Surface consistent, documented error codes so the Executor can distinguish idempotency signals from failures.



**Now, let's close the loop**



# Collector



NAR Reference Architecture — Minimal

*The Collector retrieves (reads) that state from devices using push or pull protocols.*

# Collector - Capabilities

## MUST

Include capabilities for retrieving live data using read interfaces (push and pull).

Inventory



Collector

PUSH



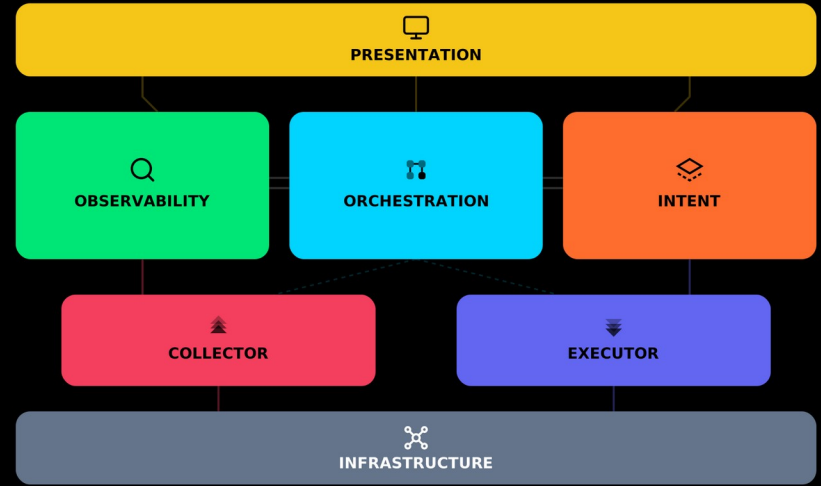
PULL



Infrastructure



# Observability



*Observability stores the actual network state and defines the logic to process observed data.*

# Observability - Capabilities

## **MUST**

Offer programmatic access to observability data.

## **MUST**

Support historical data persistence.

## **SHOULD**

Automatically generate events when discrepancies are detected.

## **SHOULD**

Offer a capable query language to extract data.

## **SHOULD**

Normalise data into a vendor-agnostic data model.

## **SHOULD**

Expose relevant insights into the current network state.

## **MAY**

Be enriched with contextual information from the intended state.



# Observability Data Sources

---

**Logs / Events**

**Metrics**

**Traces**

**Flows**

**Packet  
Captures**

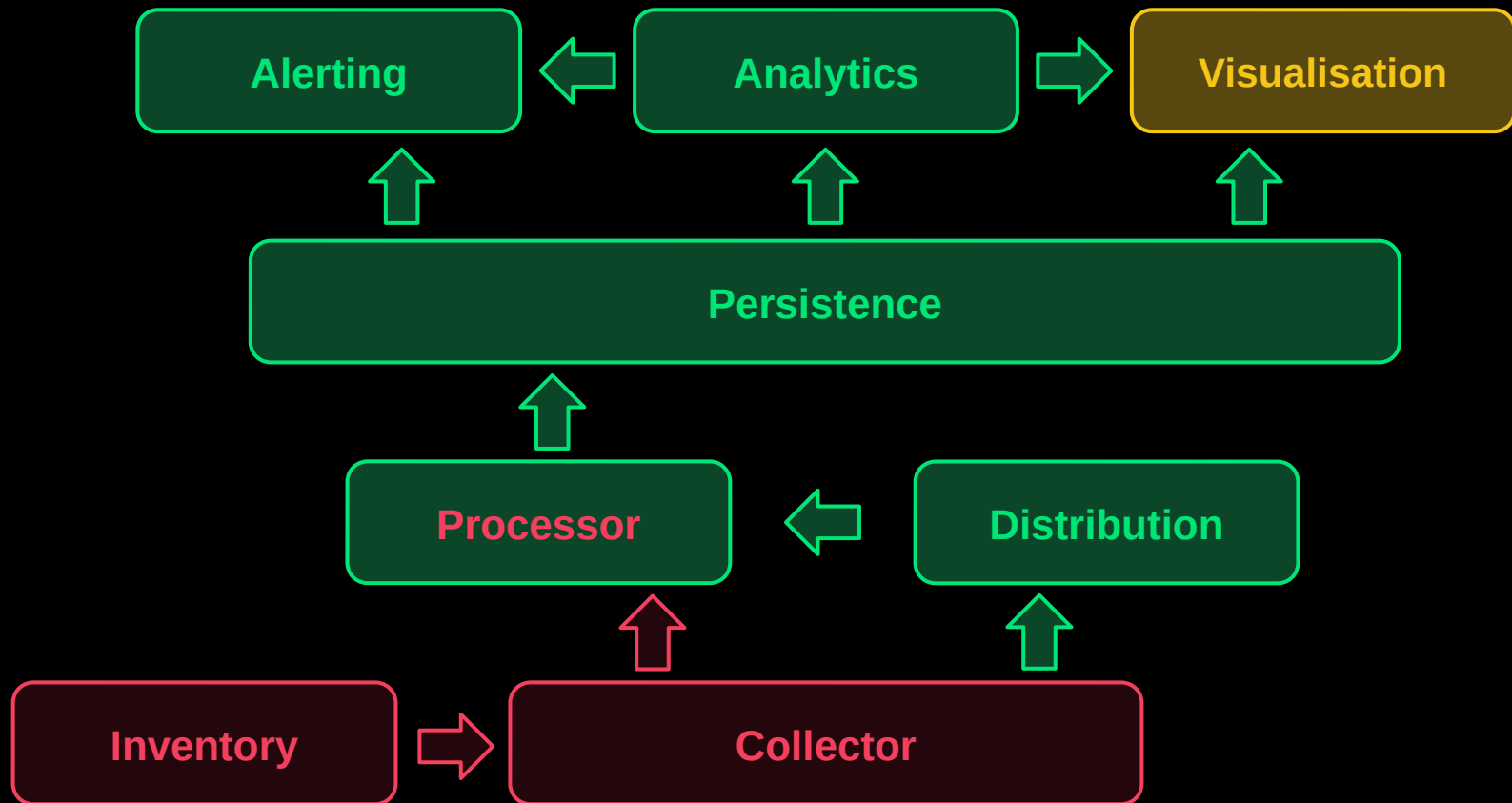
**State**

**Config**

**Probes**



# Observability - Functionalities



# Observability - Functionalities (with description)

## Visualisation

Dashboards for network state, service health, and automation system observability.

## Alerting

Threshold rules, event generation, alert-to-Orchestrator interface.

## Analytics

Anomaly detection, drift comparison against Intent; AIOps correlation; alert-to-Orchestrator interface.

## Persistence

Time-series (metrics), structured log (events), document (state snapshots); retention policies per data type.

## Distribution

Fan-out collected data to time-series databases, log stores, streaming pipelines, and downstream consumers.

## Processor

Normalisation, enrichment (adds Intent context), transformation, filtering, and sampling/throttling.

## Collector

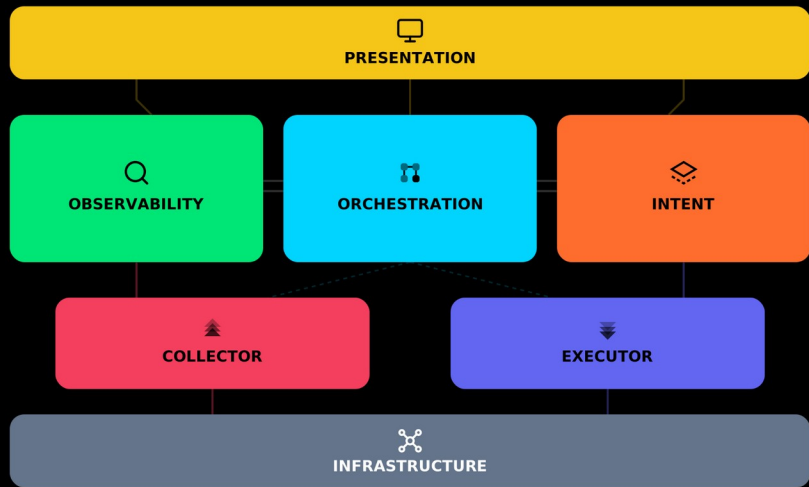
SSH, SNMP, gNMI streaming telemetry, Syslog, NETCONF, flow records, API polling; push and pull modes.

## Inventory

Collection targets sourced from the Intent; dynamic updates as the fleet changes.



# Orchestrator



NLP Reference Architecture -- Minimal

*The block that coordinates all the other blocks, deciding when each should act, sequencing multi-step workflows, handling failures gracefully, and providing tamper-evident visibility of every decision.*



# Orchestrator - Capabilities

## **MUST**

Enable coordination of processes across the various building blocks.

## **SHOULD**

Follow an event-driven approach for process execution.

## **SHOULD**

Allow for reverse/compensating actions in execution logic.

## **MAY**

Include logic to correlate multiple events and infer relationships.

## **SHOULD**

Provide a dry-run operation to check the expected workflow result.

## **SHOULD**

Provide logging and traceability of past and current workflows.

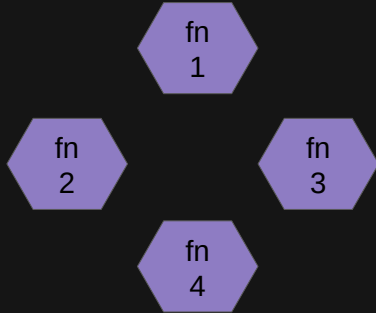
## **SHOULD**

Provide the ability to schedule the execution of a workflow.



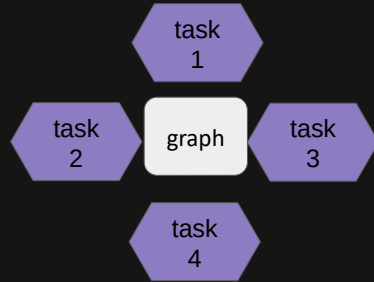
# Orchestration: Implementation approaches

## Monolith



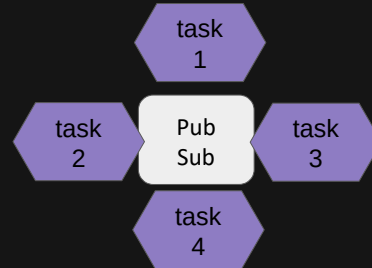
E.g. a python program

## Workflow



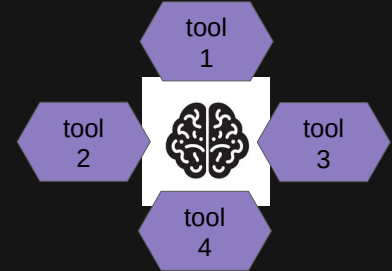
Ansible (predefined workflow)

## Choreography



Event driven and distributed

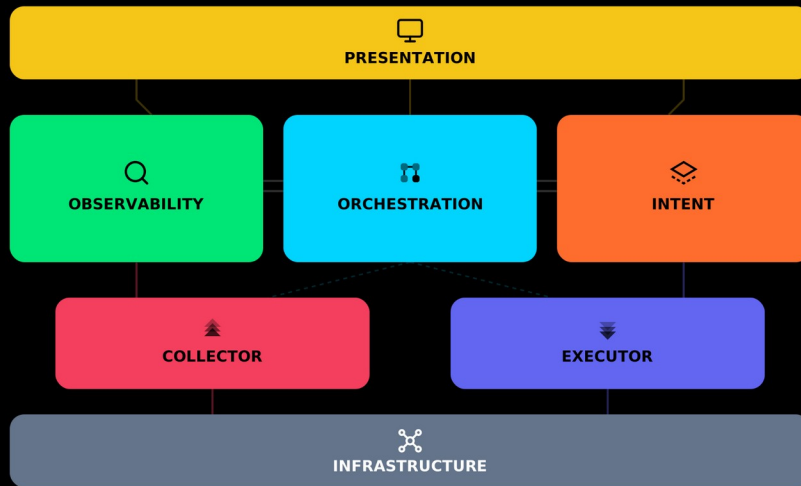
## Agentic



Agentic



# Presentation



NAP Reference Architecture — Minimal

*The outward-facing layer that makes the automation platform accessible to audiences who need not understand its internals: application teams, auditors, CI/CD pipelines, and external systems.*



# Presentation - Capabilities

---

## MUST

Provide robust authentication and authorisation capabilities.

## MAY

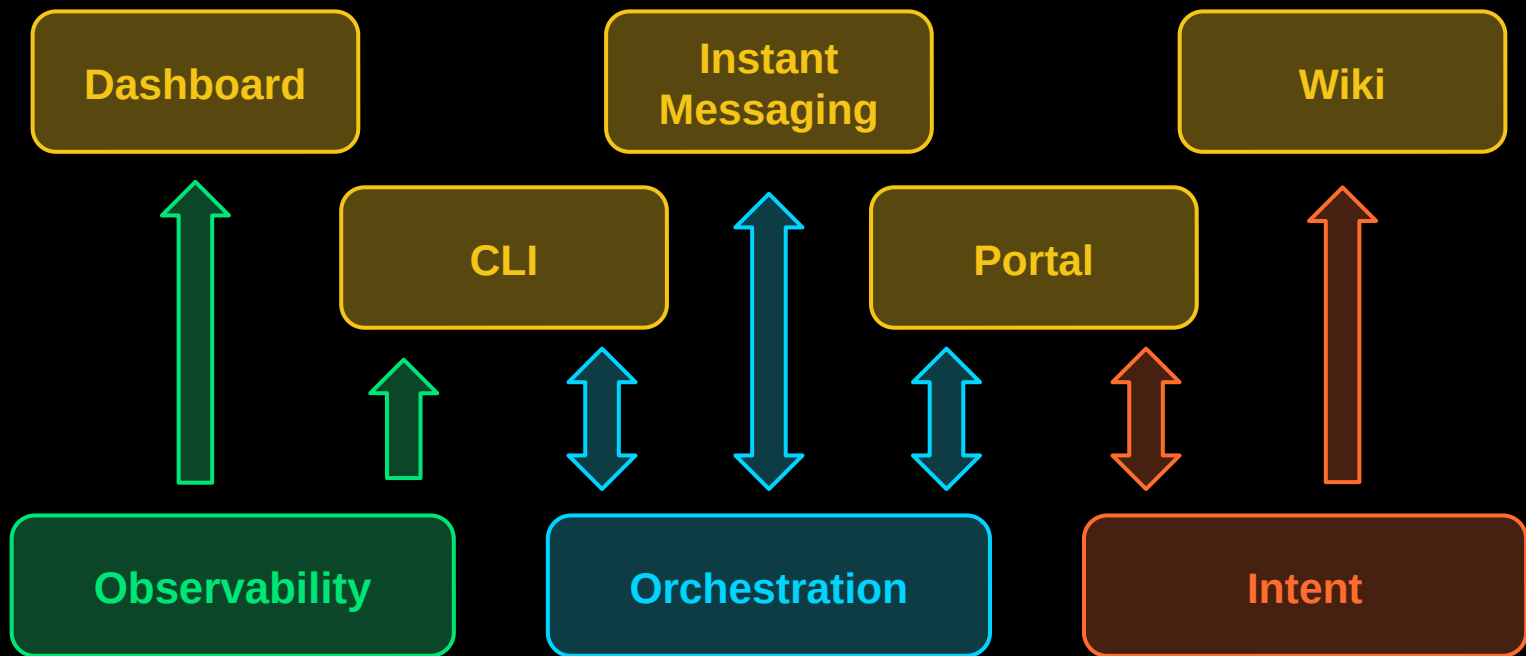
Support both read and write interactions.

## MAY

Take various forms depending on the needs of the end user.



# Presentation - Interfaces



# Additional Resources



# Additional Resources

## Designing Network Automation at Scale

A comprehensive guide to designing scalable, reliable, and maintainable network automation systems

<https://designingnetworkautomation.com/>  
By Christian Adell

## NAF Network Automation Solution Wizard

This application helps you apply the Network Automation Forum's NAF to your automation projects.

<https://naf-naf-wizard.streamlit.app/>  
By Claudia de Luna



# Summary

---

- Not a standard, it's a guide
- You don't need everything to get started
- Think Automation as a system
- Focus on functional first, tools later
- Each tool can map to multiple building blocks
- Don't reinvent the wheel
- Map your existing workflow toward the framework
- Give us some honest feedback good and bad



# How can you help

- Use case document Github:
  - <https://github.com/Network-Automation-Forum/reference/blob/main/docs/Framework/Framework.md>
- Tool mapping
- Expand details per building block
- NAF Slack channel:
  - <https://networkautomationfrm.slack.com/archives/C082G6VGZHP>



# Questions ?



# Thank You

